

TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue ETL pipeline. It defines the technical architecture, data structures, processing logic, and operational parameters required to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytical aggregations.

Scope

This TRD provides technical specifications for:

- AWS Glue job configuration and runtime parameters
- Source and target data structures with schema definitions
- Data quality and transformation logic implementation details
- Apache Hudi configuration for SCD Type 2 implementation
- AWS Glue Data Catalog integration specifications
- Error handling and logging strategies
- Incremental processing mechanisms
- Aggregation computation logic

This TRD does NOT cover:

- AWS infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy definitions
- AWS Glue job scheduling or orchestration configuration
- Cost optimization strategies
- Disaster recovery procedures
- Data retention policy implementation

Assumptions

1. Data Format Assumptions:

- Customer CSV schema: CustId (String), Name (String), EmailId (String), Region (String)

- Order CSV schema: OrderId (String), ItemName (String), PricePerUnit (Decimal), Qty (Integer), Date (String in parseable format), CustId (String)
- CSV files use comma delimiter with header row
- Date format in order data is ISO 8601 compatible (YYYY-MM-DD)

2. Environment Assumptions:

- AWS Glue version 4.0 or higher with Python 3.10 runtime
- Apache Hudi 0.12.x or higher libraries available
- Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` pre-exists
- S3 bucket `adif-sdlc` exists with appropriate lifecycle policies
- AWS Glue job has IAM permissions for S3 read/write and Glue Catalog operations

3. Processing Assumptions:

- "Null" string literal and actual NULL values are the only data quality issues
- Deduplication considers all columns for uniqueness determination
- Customer change detection compares current ingestion against cataloged customer table
- SCD2 business key for ordersummary is CustId + OrderId
- TotalAmount precision is Decimal(18,2)
- OpTs uses UTC timezone for all timestamp operations

4. Performance Assumptions:

- Dataset sizes support single Glue job execution (no distributed processing required)
- Hudi write operation uses UPSERT mode with default parallelism
- Parquet file format for all curated and analytics outputs
- No partitioning strategy specified; assume non-partitioned tables

5. Operational Assumptions:

- CloudWatch Logs retention period follows AWS account defaults
- Job bookmark not required (full refresh pattern for source ingestion)
- Athena queries will use Hudi snapshot query mode for ordersummary table
-

2. Technical Requirements

TR-INGEST-001: Customer Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 source location into a Spark DataFrame for downstream processing.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- Source Format: CSV with header
- Source Pattern: All .csv files in the directory
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: customer_raw_df
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:
 1. Initialize Spark session with AWS Glue context
 2. Use Spark CSV reader with options:
 - header=true
 - inferSchema=false (explicit schema definition)
 - mode=FAILFAST
 3. Define explicit schema matching Input Schema
 4. Read all CSV files from source path into DataFrame
 5. Validate DataFrame is not empty
 6. Return DataFrame for next processing stage
- Validation Rules:
 - Source path must exist and be accessible
 - At least one CSV file must be present
 - All CSV files must conform to expected schema
 - No schema drift allowed between files

- CustId must not be empty string
- Error Handling and Logging:
 - Log INFO: Start of ingestion with source path and file count
 - Log ERROR: If source path does not exist, raise exception with path details
 - Log ERROR: If schema mismatch detected, raise exception with expected vs actual schema
 - Log ERROR: If no files found, raise exception
 - Log WARN: If any file is empty, log filename and continue
 - Log INFO: Successful ingestion with record count
- Runtime Parameters:
 - `source\_customer\_path`: s3://adif-sdlc/sdlc_wizard/customerdata/
 - `csv\_delimiter`: ","
 - `csv\_header`: "true"
 - `csv\_quote\_char`: "\""
- Operational Notes:
 - Expected execution time: < 1 minute for datasets up to 1M records
 - Memory allocation: Standard Glue worker (G.1X) sufficient
 - No job bookmark required (full refresh pattern)
-

TR-INGEST-002: Order Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order CSV files from S3 source location into a Spark DataFrame for downstream processing.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
 - Source Format: CSV with header
 - Source Pattern: All .csv files in the directory
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: order_raw_df
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Initialize Spark session with AWS Glue context
2. Use Spark CSV reader with options:
 - header=true
 - inferSchema=false (explicit schema definition)
 - mode=FAILFAST
3. Define explicit schema matching Input Schema
4. Read all CSV files from source path into DataFrame
5. Cast PricePerUnit to Decimal(18,2)
6. Cast Qty to Integer
7. Validate DataFrame is not empty
8. Return DataFrame for next processing stage

- Validation Rules:

- Source path must exist and be accessible
- At least one CSV file must be present
- All CSV files must conform to expected schema
- OrderId must not be empty string
- PricePerUnit must be numeric and ≥ 0
- Qty must be integer and > 0
- Date must be parseable to date type

- Error Handling and Logging:

- Log INFO: Start of ingestion with source path and file count
- Log ERROR: If source path does not exist, raise exception with path details
- Log ERROR: If schema mismatch detected, raise exception with expected vs actual schema

- Log ERROR: If no files found, raise exception
- Log ERROR: If PricePerUnit or Qty casting fails, raise exception with invalid records
- Log WARN: If any file is empty, log filename and continue
- Log INFO: Successful ingestion with record count
- Runtime Parameters:
 - `source\_order\_path`: s3://adif-sdlc/sdlc_wizard/orderdata/
 - `csv\_delimiter`: ","
 - `csv\_header`: "true"
 - `csv\_quote\_char`: "\""
- Operational Notes:
 - Expected execution time: < 2 minutes for datasets up to 10M records
 - Memory allocation: Standard Glue worker (G.1X) sufficient
 - No job bookmark required (full refresh pattern)
-

TR-CLEAN-001: Customer Data Quality Processing

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove records containing "Null" string values, NULL values, and duplicate records from customer dataset to ensure data quality.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: customer_raw_df (output from TR-INGEST-001)
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: customer_clean_df
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (cleaned DataFrame)

- Processing / Transformation Logic:

1. Accept customer_raw_df as input

2. Filter out rows where any column contains string literal "Null" (case-insensitive):

- Apply filter: NOT (CustId == "Null" OR Name == "Null" OR EmailId == "Null" OR Region == "Null")

3. Filter out rows where any column is NULL:

- Apply filter: CustId IS NOT NULL AND Name IS NOT NULL AND EmailId IS NOT NULL AND Region IS NOT NULL

4. Remove duplicate records:

- Use dropDuplicates() on all columns [CustId, Name, EmailId, Region]

5. Count records before and after cleaning

6. Return cleaned DataFrame

- Validation Rules:

- At least one record must remain after cleaning

- No duplicate CustId values in output

- All output columns must be non-null

- No string literal "Null" in any column

- Error Handling and Logging:

- Log INFO: Start of cleaning with input record count

- Log WARN: Number of records removed due to "Null" string

- Log WARN: Number of records removed due to NULL values

- Log WARN: Number of duplicate records removed

- Log ERROR: If all records removed, raise exception with data quality failure message

- Log INFO: Successful cleaning with output record count and removal statistics

- Runtime Parameters:

- `null\_string\_pattern`: "Null" (case-insensitive)

- `min\_output\_records`: 1

- Operational Notes:

- Expected execution time: < 30 seconds for datasets up to 1M records

- Deduplication is deterministic (first occurrence retained)

- Memory overhead for deduplication: ~2x input dataset size

-

TR-CLEAN-002: Order Data Quality Processing

- Related Functional Requirement ID(s): FR-CLEAN-002
- Objective:

Remove records containing "Null" string values, NULL values, and duplicate records from order dataset to ensure data quality.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: order_raw_df (output from TR-INGEST-002)
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: order_clean_df
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (cleaned DataFrame)

- Processing / Transformation Logic:
 1. Accept order_raw_df as input
 2. Filter out rows where any column contains string literal "Null" (case-insensitive):
 - Apply filter: NOT (OrderId == "Null" OR ItemName == "Null" OR Date == "Null" OR CustId == "Null")
 3. Filter out rows where any column is NULL:
 - Apply filter: OrderId IS NOT NULL AND ItemName IS NOT NULL AND PricePerUnit IS NOT NULL AND Qty IS NOT NULL AND Date IS NOT NULL AND CustId IS NOT NULL
 4. Remove duplicate records:
 - Use dropDuplicates() on all columns [OrderId, ItemName, PricePerUnit, Qty, Date, CustId]

5. Count records before and after cleaning

6. Return cleaned DataFrame

- Validation Rules:

- At least one record must remain after cleaning
- No duplicate OrderId values in output
- All output columns must be non-null
- No string literal "Null" in any column
- PricePerUnit must be ≥ 0
- Qty must be > 0
- Error Handling and Logging:
 - Log INFO: Start of cleaning with input record count
 - Log WARN: Number of records removed due to "Null" string
 - Log WARN: Number of records removed due to NULL values
 - Log WARN: Number of duplicate records removed
 - Log ERROR: If all records removed, raise exception with data quality failure message
 - Log INFO: Successful cleaning with output record count and removal statistics
- Runtime Parameters:
 - ``null_string_pattern``: "Null" (case-insensitive)
 - ``min_output_records``: 1
- Operational Notes:
 - Expected execution time: < 1 minute for datasets up to 10M records
 - Deduplication is deterministic (first occurrence retained)
 - Memory overhead for deduplication: $\sim 2x$ input dataset size
-

TR-CATALOG-001: Customer Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrame

- Source Name: customer_clean_df (output from TR-CLEAN-001)
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_customer
- Target S3 Path: s3:///glue_catalog/sdlc_wizard_customer/
- Target Format: Parquet
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (Parquet table)

- Processing / Transformation Logic:
 1. Accept customer_clean_df as input
 2. Write DataFrame to S3 using Parquet format:
 - Mode: overwrite
 - Compression: snappy
 - Path: s3:///glue_catalog/sdlc_wizard_customer/
 3. Register table in Glue Data Catalog:
 - Database: gen_ai_poc_databrickscoe
 - Table: sdlc_wizard_customer
 - Format: parquet
 - Location: s3:///glue_catalog/sdlc_wizard_customer/
 4. Verify table registration successful
 5. Log table metadata (record count, file size, partition info)
- Validation Rules:
 - Target database must exist in Glue Data Catalog
 - Write operation must complete successfully
 - Table must be queryable via Athena after registration

- Schema in catalog must match DataFrame schema
- Error Handling and Logging:
 - Log INFO: Start of catalog registration with target database and table
 - Log ERROR: If database does not exist, raise exception with database name
 - Log ERROR: If S3 write fails, raise exception with S3 path and error details
 - Log ERROR: If catalog registration fails, raise exception with catalog error
 - Log INFO: Successful registration with table location and record count
- Runtime Parameters:
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_table`: sdlc_wizard_customer
 - `target\_s3\_path`: s3:///glue_catalog/sdlc_wizard_customer/
 - `write\_mode`: overwrite
 - `compression`: snappy
- Operational Notes:
 - Expected execution time: < 1 minute for datasets up to 1M records
 - Parquet file size target: 128MB per file
 - No partitioning applied
 - Table supports full table scan queries in Athena
-

TR-CATALOG-002: Order Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned order data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: order_clean_df (output from TR-CLEAN-002)
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: sdlc_wizard_order

- Target S3 Path: s3:///glue_catalog/sdlc_wizard_order/
- Target Format: Parquet
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (Parquet table)

- Processing / Transformation Logic:
 1. Accept order_clean_df as input
 2. Write DataFrame to S3 using Parquet format:
 - Mode: overwrite
 - Compression: snappy
 - Path: s3:///glue_catalog/sdlc_wizard_order/
 3. Register table in Glue Data Catalog:
 - Database: gen_ai_poc_databrickscoe
 - Table: sdlc_wizard_order
 - Format: parquet
 - Location: s3:///glue_catalog/sdlc_wizard_order/
 4. Verify table registration successful
 5. Log table metadata (record count, file size, partition info)
- Validation Rules:
 - Target database must exist in Glue Data Catalog
 - Write operation must complete successfully
 - Table must be queryable via Athena after registration
 - Schema in catalog must match DataFrame schema
- Error Handling and Logging:
 - Log INFO: Start of catalog registration with target database and table

- Log ERROR: If database does not exist, raise exception with database name
- Log ERROR: If S3 write fails, raise exception with S3 path and error details
- Log ERROR: If catalog registration fails, raise exception with catalog error
- Log INFO: Successful registration with table location and record count
- Runtime Parameters:
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_table`: sdlc_wizard_order
 - `target\_s3\_path`: s3:///glue_catalog/sdlc_wizard_order/
 - `write\_mode`: overwrite
 - `compression`: snappy
- Operational Notes:
 - Expected execution time: < 2 minutes for datasets up to 10M records
 - Parquet file size target: 128MB per file
 - No partitioning applied
 - Table supports full table scan queries in Athena
-

TR-SCD2-001: Order Summary SCD Type 2 Implementation

- Related Functional Requirement ID(s): FR-SCD2-001
- Objective:

Join customer and order data, implement SCD Type 2 logic using Apache Hudi to maintain historical order summary records with active/inactive tracking.

- Source Details:
 - Source Type 1: In-memory Spark DataFrame
 - Source Name 1: customer_clean_df (output from TR-CLEAN-001)
 - Source Type 2: In-memory Spark DataFrame
 - Source Name 2: order_clean_df (output from TR-CLEAN-002)
 - Source Type 3: Apache Hudi Table (if exists)
 - Source Path 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
 - Target Type: Apache Hudi Table (Copy-on-Write)
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: ordersummary

- Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Format: Hudi
- Input Schema (customer_clean_df):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Input Schema (order_clean_df):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from customer_clean_df.CustId	
	Name	String	Variable	No	Direct map from customer_clean_df.Name	
	EmailId	String	Variable	No	Direct map from customer_clean_df.EmailId	
	Region	String	Variable	No	Direct map from customer_clean_df.Region	
	OrderId	String	Variable	No	Direct map from order_clean_df.OrderId	
	ItemName	String	Variable	No	Direct map from order_clean_df.ItemName	
	PricePerUnit	Decimal	18,2	No	Direct map from order_clean_df.PricePerUnit	
	Qty	Integer	-	No	Direct map from order_clean_df.Qty	
	Date	String	Variable	No	Direct map from order_clean_df.Date	
	IsActive	Boolean	-	No	Derived: true for new/updated records	
	StartDate	Timestamp	-	No	Derived: current_timestamp() for new/updated records	
	EndDate	Timestamp	-	Yes	Derived: null for active records, current_timestamp() for closed records	
	OpTs	Timestamp	-	No	Derived: current_timestamp() for all operations	

- Processing / Transformation Logic:

1. Join Operation:

- Perform INNER JOIN between customer_clean_df and order_clean_df on CustId
- Result: joined_df with all customer and order columns

2. Read Existing Hudi Table (if exists):

- Check if s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ exists
- If exists, read Hudi table into existing_ordersummary_df
- If not exists, create empty DataFrame with output schema

3. Identify Changes:

- Business key: CustId + OrderId
- Compare joined_df against existing_ordersummary_df where IsActive = true
- Identify new records: records in joined_df not in existing active records
- Identify changed records: records in joined_df with different values than existing active records
- Identify unchanged records: records in joined_df matching existing active records

4. SCD2 Update Logic:

- For unchanged records: No action required
- For changed records:
 - Create close_records_df: Copy existing active records, set IsActive = false, EndDate = current_timestamp()
 - Create new_active_records_df: Copy from joined_df, set IsActive = true, StartDate = current_timestamp(), EndDate = null
- For new records:
 - Create new_records_df: Copy from joined_df, set IsActive = true, StartDate = current_timestamp(), EndDate = null

5. Combine Updates:

- Union close_records_df, new_active_records_df, and new_records_df
- Add OpTs = current_timestamp() to all records
- Result: updates_df

6. Hudi Write Operation:

- Configure Hudi options:
 - Table type: COPY_ON_WRITE
 - Operation: UPSERT
 - Record key: CustId + OrderId
 - Precombine field: OpTs
 - Partition path: (none - non-partitioned)
- Write updates_df to s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Hudi will handle merge logic based on record key and precombine field

7. Catalog Registration:

- Register Hudi table in Glue Data Catalog
- Database: gen_ai_poc_databricksco
- Table: ordersummary
- Configure Hudi-specific properties for Athena compatibility
- Validation Rules:
 - Join must produce at least one record
 - No duplicate active records for same business key (CustId + OrderId)
 - All active records must have IsActive = true, EndDate = null
 - All inactive records must have IsActive = false, EndDate IS NOT NULL
 - StartDate must be <= EndDate for inactive records
 - OpTs must be populated for all records
- Error Handling and Logging:
 - Log INFO: Start of SCD2 processing with input record counts
 - Log INFO: Join result record count
 - Log INFO: Number of new, changed, and unchanged records identified
 - Log WARN: If join produces no results, log warning and create empty Hudi table
 - Log ERROR: If Hudi write fails, raise exception with Hudi error details
 - Log ERROR: If duplicate active records detected, raise exception with business key details
 - Log INFO: Successful SCD2 update with final record counts (active/inactive)
- Runtime Parameters:
 - `hudi\_table\_name`: ordersummary
 - `hudi\_table\_type`: COPY_ON_WRITE
 - `hudi\_operation`: upsert
 - `hudi\_record\_key`: CustId,OrderId
 - `hudi\_precombine\_field`: OpTs
 - `hudi\_base\_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - `target\_database`: gen_ai_poc_databricksco
 - `target\_table`: ordersummary
- Operational Notes:
 - Expected execution time: < 5 minutes for datasets up to 10M records
 - Hudi COPY_ON_WRITE provides snapshot isolation for queries

- Athena queries use snapshot query mode by default
- Time-travel queries supported via Hudi timestamp parameters
- Memory allocation: G.2X worker recommended for large datasets
-

TR-INCR-001: Incremental Customer Update Processing

- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Detect changes in customer data and trigger SCD Type 2 updates to ordersummary table for affected customer records.

- Source Details:
- Source Type 1: In-memory Spark DataFrame
- Source Name 1: customer_clean_df (output from TR-CLEAN-001)
- Source Type 2: AWS Glue Data Catalog Table
- Source Database 2: gen_ai_poc_databrickscoe
- Source Table 2: sdlc_wizard_customer
- Source Type 3: Apache Hudi Table
- Source Path 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
- Target Type: Apache Hudi Table (Copy-on-Write)
- Target Database: gen_ai_poc_databrickscoe
- Target Table: ordersummary
- Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Input Schema (customer_clean_df):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Input Schema (sdlc_wizard_customer - cataloged):

Same as customer_clean_df

- Output Schema:

Same as TR-SCD2-001 Output Schema (ordersummary table)

- Processing / Transformation Logic:

1. Read Cataloged Customer Data:

- Read sdlc_wizard_customer table from Glue Data Catalog
- Result: cataloged_customer_df

2. Identify Changed Customers:

- Perform FULL OUTER JOIN between customer_clean_df and cataloged_customer_df on CustId
- Identify new customers: CustId in customer_clean_df but not in cataloged_customer_df
- Identify changed customers: CustId in both, but Name, EmailId, or Region values differ
- Identify deleted customers: CustId in cataloged_customer_df but not in customer_clean_df (optional handling)
- Result: changed_customers_df containing CustId of affected customers

3. Filter Affected Order Summary Records:

- Read ordersummary Hudi table where IsActive = true
- Filter records where CustId IN (changed_customers_df.CustId)
- Result: affected_ordersummary_df

4. Apply SCD2 Update Logic:

- For each affected order summary record:
- Close existing active record: Set IsActive = false, EndDate = current_timestamp()
- Create new active record with updated customer information from customer_clean_df
- Set IsActive = true, StartDate = current_timestamp(), EndDate = null
- Set OpTs = current_timestamp()

5. Hudi Write Operation:

- Use same Hudi configuration as TR-SCD2-001
- Write updated records to ordersummary table
- Hudi UPSERT will handle merge logic

6. Update Customer Catalog Table:

- Overwrite sdlc_wizard_customer table with customer_clean_df
- This becomes the new baseline for next incremental run

- Validation Rules:

- Changed customers list must not contain duplicates
- All affected order summary records must be updated

- No orphaned active records after update
- Customer catalog table must be updated successfully
- Error Handling and Logging:
 - Log INFO: Start of incremental processing with input record counts
 - Log INFO: Number of new, changed, and deleted customers identified
 - Log INFO: Number of affected order summary records
 - Log WARN: If no customer changes detected, log message and skip SCD2 update
 - Log ERROR: If change detection fails, raise exception with comparison error details
 - Log ERROR: If Hudi write fails, raise exception with Hudi error details
 - Log INFO: Successful incremental update with affected record counts
- Runtime Parameters:
 - `change\_detection\_columns`: Name,EmailId,Region
 - `hudi\_table\_name`: ordersummary
 - `hudi\_operation`: upsert
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_table`: ordersummary
- Operational Notes:
 - Expected execution time: < 3 minutes for typical incremental changes
 - Change detection compares all non-key customer attributes
 - Deleted customers do not trigger ordersummary updates (assumption)
 - This requirement assumes incremental pattern; if full refresh is used, this step may be skipped
-

TR-AGG-001: Customer Aggregate Spend Calculation

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Join customer and order data, calculate total spending per customer per date, and produce aggregated analytics table.

- Source Details:
 - Source Type 1: In-memory Spark DataFrame
 - Source Name 1: customer_clean_df (output from TR-CLEAN-001)
 - Source Type 2: In-memory Spark DataFrame
 - Source Name 2: order_clean_df (output from TR-CLEAN-002)

- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databricksco
- Target Table: customeraggregatespend
- Target S3 Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Target Format: Parquet

- Input Schema (customer_clean_df):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Input Schema (order_clean_df):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	Name	String	Variable	No	Direct map from customer_clean_df.Name	
	TotalAmount	Decimal	18,2	No	Derived: SUM(order_clean_df.PricePerUnit × order_clean_df.Qty)	
	Date	String	Variable	No	Direct map from order_clean_df.Date	

- Processing / Transformation Logic:

1. Join Operation:

- Perform INNER JOIN between customer_clean_df and order_clean_df on CustId
- Result: joined_df with customer Name and all order columns

2. Calculate TotalAmount:

- Add calculated column: TotalAmount = PricePerUnit × Qty

- Cast result to Decimal(18,2)

3. Aggregate by Customer and Date:

- Group by: Name, Date
- Aggregate: SUM(TotalAmount) AS TotalAmount
- Result: aggregated_df with columns [Name, TotalAmount, Date]

4. Write to S3:

- Write aggregated_df to s3://adif-sdlc/analytics/customeragggregatespend/
- Format: Parquet
- Mode: overwrite
- Compression: snappy

5. Register in Glue Data Catalog:

- Database: gen_ai_poc_databrickscoe
- Table: customeragggregatespend
- Location: s3://adif-sdlc/analytics/customeragggregatespend/
- Validation Rules:
 - Join must produce at least one record (or create empty table with schema)
 - TotalAmount must be ≥ 0 for all records
 - No duplicate Name + Date combinations in output
 - All output columns must be non-null
- Error Handling and Logging:
 - Log INFO: Start of aggregation with input record counts
 - Log INFO: Join result record count
 - Log WARN: If join produces no results, create empty Parquet table with schema
 - Log ERROR: If TotalAmount calculation produces negative values, raise exception
 - Log ERROR: If S3 write fails, raise exception with S3 path and error details
 - Log ERROR: If catalog registration fails, raise exception with catalog error
 - Log INFO: Successful aggregation with output record count and total spend sum
- Runtime Parameters:
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_table`: customeragggregatespend
 - `target\_s3\_path`: s3://adif-sdlc/analytics/customeragggregatespend/
 - `write\_mode`: overwrite

- `compression`: snappy
- Operational Notes:
 - Expected execution time: < 2 minutes for datasets up to 10M orders
 - Aggregation uses Spark's built-in groupBy and sum functions
 - No partitioning applied to analytics table
 - Table optimized for analytical queries on customer spending patterns
-

TR-CATALOG-003: Order Summary Hudi Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-003
- Objective:

Register ordersummary Hudi table in AWS Glue Data Catalog with Hudi-specific properties for Athena query compatibility.

- Source Details:
 - Source Type: Apache Hudi Table
 - Source Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: ordersummary
 - Target Format: Hudi

- Input Schema:

Same as TR-SCD2-001 Output Schema

- Output Schema:

Same as Input Schema (Glue Catalog metadata)

- Processing / Transformation Logic:
 1. Verify Hudi Table Exists:
 - Check s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ contains Hudi metadata
 - Verify .hoodie directory exists
 2. Register Hudi Table in Glue Catalog:
 - Use Hudi sync tool or Spark Hudi DataSource writer options
 - Set catalog sync properties:

- `hoodie.datasource.hive_sync.enable = true`
- `hoodie.datasource.hive_sync.database = gen_ai_poc_databrickscoe`
- `hoodie.datasource.hive_sync.table = ordersummary`
- `hoodie.datasource.hive_sync.mode = hms`
- `hoodie.datasource.hive_sync.use_jdbc = false`

3. Configure Athena Compatibility:

- Set table properties for Athena Hudi support:
- `EXTERNAL = TRUE`
- `table_type = COPY_ON_WRITE`
- `hoodie.query.type = snapshot`

4. Verify Registration:

- Query Glue Data Catalog to confirm table exists
- Verify schema matches Hudi table schema
- Validation Rules:
- Hudi table must exist at specified S3 path
- Glue Data Catalog database must exist
- Table registration must complete successfully
- Table must be queryable via Athena
- Error Handling and Logging:
- Log INFO: Start of Hudi catalog registration
- Log ERROR: If Hudi table does not exist, raise exception with S3 path
- Log ERROR: If database does not exist, raise exception with database name
- Log ERROR: If Hudi sync fails, raise exception with sync error details
- Log INFO: Successful registration with table location and schema
- Runtime Parameters:
- ``hoodie_base_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
- ``target_database`: gen_ai_poc_databrickscoe`
- ``target_table`: ordersummary`
- ``hoodie_sync_mode`: hms`
- Operational Notes:
- Hudi sync occurs automatically during write operation if configured
- Manual sync may be required if write and catalog registration are separate steps

- Athena queries use snapshot mode by default
- Time-travel queries require specific Hudi query parameters
-

TR-CATALOG-004: Customer Aggregate Spend Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-004
- Objective:

Register customeraggregatespend analytics table in AWS Glue Data Catalog for Athena query access.

- Source Details:
- Source Type: Parquet Files
- Source Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: customeraggregatespend
- Target Format: Parquet
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	Name	String	Variable	No	
	TotalAmount	Decimal	18,2	No	
	Date	String	Variable	No	

- Output Schema:

Same as Input Schema (Glue Catalog metadata)

- Processing / Transformation Logic:
 1. Verify Parquet Files Exist:
 - Check s3://adif-sdlc/analytics/customeraggregatespend/ contains Parquet files
 2. Register Table in Glue Data Catalog:
 - Use Spark DataFrameWriter with catalog sync enabled
 - Or use AWS Glue Crawler to auto-detect schema
 - Database: gen_ai_poc_databrickscoe
 - Table: customeraggregatespend

- Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Format: parquet

3. Verify Registration:

- Query Glue Data Catalog to confirm table exists
- Verify schema matches Parquet file schema
- Validation Rules:
 - Parquet files must exist at specified S3 path
 - Glue Data Catalog database must exist
 - Table registration must complete successfully
 - Table must be queryable via Athena
- Error Handling and Logging:
 - Log INFO: Start of catalog registration
 - Log ERROR: If Parquet files do not exist, raise exception with S3 path
 - Log ERROR: If database does not exist, raise exception with database name
 - Log ERROR: If catalog registration fails, raise exception with catalog error
 - Log INFO: Successful registration with table location and schema
- Runtime Parameters:
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_table`: customeraggregatespend
 - `target\_s3\_path`: s3://adif-sdlc/analytics/customeraggregatespend/
 - `table\_format`: parquet
- Operational Notes:
 - Catalog registration occurs automatically during Parquet write if configured
 - Manual registration via Glue Crawler is alternative approach
 - Table supports full table scan queries in Athena
 - No partitioning applied; consider partitioning by Date for large datasets
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python 3.10
- Framework: PySpark 3.3.x (AWS Glue 4.0)
- Additional Libraries:

- Apache Hudi 0.12.x (for SCD2 implementation)
- AWS Glue Python libraries (awsglue.context, awsglue.dynamicframe)

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Execution Pattern: Batch processing with full refresh for source ingestion
- Incremental Processing: Supported via TR-INCR-001 for customer updates
- Parallelism: Spark default parallelism based on worker count
- Job Bookmark: Not required (full refresh pattern for CSV sources)

Glue Job Type Expectations

- Worker Type: G.1X (4 vCPU, 16 GB memory) for standard workloads
- Worker Type: G.2X (8 vCPU, 32 GB memory) for large datasets or Hudi operations
- Number of Workers: 2-10 based on dataset size
- Timeout: 2880 minutes (48 hours maximum)
- Max Retries: 0 (fail fast for data quality issues)
- Glue Version: 4.0
- Python Version: 3.10
-

4. Data Design Details

Source Paths / Source Systems

	Source ID	Source Type	Source Path	Format	Schema Reference	
	-----	-----	-----	-----	-----	
	SRC-001	S3 CSV	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001 Input Schema	
	SRC-002	S3 CSV	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002 Input Schema	
	SRC-003	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	TR-SCD2-001 Output Schema	
	SRC-004	Glue Catalog	gen_ai_poc_databrickscoe.sdlc_wizard_customer	Parquet	TR-CATALOG-001 Output Schema	

- Note: All source paths are preserved exactly as specified in the FRD. No intermediate sources or staging tables are introduced.

Target Paths / Target Tables

Target ID	Target Type	Target Path	Format	Schema Reference	Update Behavior
-----	-----	-----	-----	-----	-----
GT-001	Glue Catalog	gen_ai_poc_databrickscoe.sdlc_wizard_customer	Parquet	TR-CATALOG-001 Output Schema	Overwrite
GT-002	Glue Catalog	gen_ai_poc_databrickscoe.sdlc_wizard_order	Parquet	TR-CATALOG-002 Output Schema	Overwrite
GT-003	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	TR-SCD2-001 Output Schema	Upsert (SCD2)

GT-004	Glue Catalog	gen_ai_poc_databricksco.ordersummary	Hudi	TR-CATALOG-003 Output Schema	Upsert (SCD2)
GT-005	Glue Catalog	gen_ai_poc_databricksco.customeraggregatespend	Parquet	TR-AGG-001 Output Schema	Overwrite

Catalog Registration Expectations

- Database: gen_ai_poc_databricksco (must pre-exist)
- Tables: 5 tables total
- sdlc_wizard_customer (Parquet)
- sdlc_wizard_order (Parquet)
- ordersummary (Hudi)
- customeraggregatespend (Parquet)
- Registration Method: Spark DataFrameWriter with catalog sync enabled
- Athena Compatibility: All tables must be queryable via Athena
- Hudi-Specific: ordersummary table requires Hudi-specific table properties

Partitioning Expectations

- No Partitioning: All tables are non-partitioned based on FRD specifications
- Future Consideration: Date-based partitioning may be added for customeraggregatespend and ordersummary for performance optimization

File Formats / Table Formats

Table	Format	Compression	File Size Target	
-----	-----	-----	-----	
sdlc_wizard_customer	Parquet	Snappy	128 MB	
sdlc_wizard_order	Parquet	Snappy	128 MB	
ordersummary	Hudi (COPY_ON_WRITE)	Snappy	128 MB	
customeraggregatespend	Parquet	Snappy	128 MB	

Update / Overwrite / Append Behavior

Table	Write Mode	Behavior Description	
-----	-----	-----	
sdlc_wizard_customer	Overwrite	Full refresh on each run	
sdlc_wizard_order	Overwrite	Full refresh on each run	
ordersummary	Upsert	SCD Type 2 with Hudi UPSERT operation	
customeraggregatespend	Overwrite	Full refresh on each run	

- Note: No staging tables or intermediate layers are used. All processing occurs in-memory with direct writes to target locations.

•

5. Processing Details

Data Quality Rules

1. Null String Removal:

- Remove rows where any column contains string literal "Null" (case-insensitive)
- Apply to both customer and order datasets

2. NULL Value Removal:

- Remove rows where any column is NULL
- Apply to both customer and order datasets

3. Numeric Validation:

- PricePerUnit must be ≥ 0
- Qty must be > 0
- TotalAmount must be ≥ 0

4. Data Type Validation:

- PricePerUnit must be castable to Decimal(18,2)
- Qty must be castable to Integer
- Date must be parseable to date type

Deduplication Rules

1. Customer Deduplication:

- Consider all columns: CustId, Name, EmailId, Region
- Retain first occurrence (deterministic ordering)
- Apply after null removal

2. Order Deduplication:

- Consider all columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Retain first occurrence (deterministic ordering)
- Apply after null removal

Join Rules

1. Customer-Order Join (for SCD2):

- Join Type: INNER JOIN
- Join Key: CustId
- Result: All customer and order columns

2. Customer-Order Join (for Aggregation):

- Join Type: INNER JOIN
- Join Key: CustId

- Result: Customer Name and all order columns

3. Change Detection Join (for Incremental):

- Join Type: FULL OUTER JOIN
- Join Key: CustId
- Result: Identify new, changed, and deleted customers

Aggregation Rules

1. Customer Aggregate Spend:

- Group By: Name, Date
- Aggregate Function: $\text{SUM}(\text{PricePerUnit} \times \text{Qty})$
- Result Column: TotalAmount (Decimal 18,2)

SCD / History Tracking Rules

1. SCD Type 2 Implementation:

- Business Key: CustId + OrderId
- Active Record Indicator: IsActive (Boolean)
- Effective Date Range: StartDate (Timestamp), EndDate (Timestamp)
- Operation Timestamp: OpTs (Timestamp)

2. SCD2 Update Logic:

- New Record: IsActive = true, StartDate = current_timestamp(), EndDate = null
- Close Existing: IsActive = false, EndDate = current_timestamp()
- Unchanged: No action

3. Hudi Configuration:

- Table Type: COPY_ON_WRITE
- Operation: UPSERT
- Record Key: CustId, OrderId
- Precombine Field: OpTs

Null Handling

1. Source Data:

- Remove rows with NULL values in any column
- Remove rows with "Null" string literal in any column

2. SCD2 Fields:

- EndDate: NULL for active records, current_timestamp() for inactive records
- All other fields: NOT NULL

3. Aggregation:
- NULL values already removed during data quality processing
 - No additional null handling required

Type Casting Expectations

1. Order Data:
- PricePerUnit: String → Decimal(18,2)
 - Qty: String → Integer
 - Date: String → Date (for validation, stored as String)
2. SCD2 Fields:
- IsActive: Boolean
 - StartDate: Timestamp
 - EndDate: Timestamp (nullable)
 - OpTs: Timestamp
3. Aggregation:
- TotalAmount: Decimal(18,2)
 -

6. Traceability Matrix

	Technical Source

	s3://adif-sdlc/sdlc_wizard/customerdata/
	s3://adif-sdlc/sdlc_wizard/orderdata/
	customer_raw_df
	order_raw_df
	customer_clean_df
	order_clean_df
adif-sdlc/curated/sdlc_wizard/ordersummary/	customer_clean_df + order_clean_df + s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
scoe.sdlc_wizard_customer + s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	customer_clean_df + gen_ai_poc_databrickscoe.sdlc_wizard_customer + s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
	customer_clean_df + order_clean_df
mary/	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
end/	s3://adif-sdlc/analytics/customeraggregatespend/

- Source Fidelity Verification:
- All original sources from FRD are preserved exactly in TRD
- No intermediate sources, staging tables, or alternative sources introduced

- In-memory DataFrames are processing artifacts, not alternative sources
- All S3 paths match FRD specifications exactly
-

7. ETL Mapping Details

TR-INGEST-001: Customer Data Ingestion Mapping

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)
- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Target: customer_raw_df (In-memory DataFrame)
- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Column Mapping:

	Target Column	Source Column	Transformation	
	-----	-----	-----	
	CustId	CustId	Direct map	
	Name	Name	Direct map	
	EmailId	EmailId	Direct map	
	Region	Region	Direct map	

-

TR-INGEST-002: Order Data Ingestion Mapping

- Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	String	Variable	No	
	Qty	String	Variable	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: order_raw_df (In-memory DataFrame)

- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Column Mapping:

	Target Column	Source Column	Transformation	
	-----	-----	-----	
	OrderId	OrderId	Direct map	
	ItemName	ItemName	Direct map	
	PricePerUnit	PricePerUnit	Cast to Decimal(18,2)	
	Qty	Qty	Cast to Integer	
	Date	Date	Direct map	
	CustId	CustId	Direct map	

•

TR-SCD2-001: Order Summary SCD2 Mapping

- Source 1: customer_clean_df (In-memory DataFrame)

- Source 1 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Source 2: order_clean_df (In-memory DataFrame)

- Source 2 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: ordersummary (Hudi Table)

- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	IsActive	Boolean	-	No	
	StartDate	Timestamp	-	No	
	EndDate	Timestamp	-	Yes	
	OpTs	Timestamp	-	No	

- Column Mapping:

	Target Column	Source Column(s)	Transformation	
	-----	-----	-----	
	CustId	customer_clean_df.CustId	Direct map	

Name	customer_clean_df.Name	Direct map	
EmailId	customer_clean_df.EmailId	Direct map	
Region	customer_clean_df.Region	Direct map	
OrderId	order_clean_df.OrderId	Direct map	
ItemName	order_clean_df.ItemName	Direct map	
PricePerUnit	order_clean_df.PricePerUnit	Direct map	
Qty	order_clean_df.Qty	Direct map	
Date	order_clean_df.Date	Direct map	
IsActive	N/A	Derived: true for new/updated, false for closed	
StartDate	N/A	Derived: current_timestamp() for new/updated	
EndDate	N/A	Derived: null for active, current_timestamp() for closed	
OpTs	N/A	Derived: current_timestamp()	

-

TR-AGG-001: Customer Aggregate Spend Mapping

- Source 1: customer_clean_df (In-memory DataFrame)
- Source 1 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Source 2: order_clean_df (In-memory DataFrame)
- Source 2 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	18,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: customeraggregatespend (Parquet Table)
- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	Name	String	Variable	No	
	TotalAmount	Decimal	18,2	No	
	Date	String	Variable	No	

• Column Mapping:

	Target Column	Source Column(s)	Transformation	
	-----	-----	-----	
	Name	customer_clean_df.Name	Direct map (after join)	
	TotalAmount	order_clean_df.PricePerUnit, order_clean_df.Qty	Derived: SUM(PricePerUnit × Qty) grouped by Name, Date	
	Date	order_clean_df.Date	Direct map (after join)	

•

End of Technical Requirements Document